

Hai trovato il problema giusto. Ma prima di guardare a `ss`/`rr` (che danno solo un piccolo risparmio), c'è qualcosa di più grave: `algo_medium` si comporta quasi identico ad `algo_simple` (22176 vs 22178) — questo significa che il chunk-based **non sta funzionando come dovrebbe**. Con 500 numeri dovrebbe essere molto più efficiente del selection sort puro.

Il problema reale di `algo_medium`

Guarda `extract_from_chunk` — ogni volta che cerca il minimo nel chunk corrente, **scorre tutto lo stack** con `find_min_in_chunk`, poi calcola la distanza con un altro scan, poi muove. Questo è ancora $O(n)$ per estrazione, esattamente come il selection sort — il chunking ti dà beneficio solo se **dentro ogni chunk scegli l'elemento più vicino al top**, non necessariamente il rank più basso assoluto.

La vera ottimizzazione del Turk algorithm

Dentro ogni chunk, invece di prendere sempre il rank minimo, dovresti scegliere l'elemento che richiede **meno rotazioni** per arrivare in cima, tra tutti quelli che appartengono al chunk corrente. Questo riduce drasticamente il costo delle rotazioni:

```
c
static t_node *find_closest_in_chunk(t_node *a, int max_rank, int size)
{
    t_node *best;
    t_node *tmp;
    int best_dist;
    int pos;
    int dist;

    best = NULL;
    best_dist = size + 1;
    tmp = a;
    pos = 0;
    while (tmp)
    {
        if (tmp->rank < max_rank)
        {
            dist = pos;
            if (dist > size / 2)
                dist = size - dist;
            if (dist < best_dist)
                best = tmp;
        }
        tmp = tmp->next;
        pos++;
    }
    return best;
}
```

```

        {
            best = tmp;
            best_dist = dist;
        }
        tmp = tmp->next;
        pos++;
    }
    return (best);
}

```

Questo trova, tra gli elementi del chunk corrente, quello che ha la rotazione più corta — non necessariamente il valore più piccolo.

Ma il vero salto di efficienza viene da QUI

Il punto chiave del Turk algorithm reale è diverso da quello che abbiamo implementato finora: **non scorri linearmente i chunk uno dopo l'altro**. Invece, ad ogni passo, guardi solo gli elementi che sono "candidati per essere il prossimo minimo nel chunk attuale" e scegli quello più vicino al top tra **tutti i chunk non ancora completati**, non solo quello corrente. Questo è più complesso da implementare correttamente.

Per ora, soluzione pragmatica e più efficace

Prima di buttarsi sull'ottimizzazione complessa, sistemiamo la ricerca per **vicinanza** invece che per **rank minimo assoluto** — già questo dovrebbe abbassare parecchio il conteggio. Vuoi che integriamo `find_closest_in_chunk` al posto di `find_min_in_chunk` e testiamo i numeri prima di aggiungere `ss/rr`?

Q: Vuoi prima testare `find_closest_in_chunk` (fix più semplice), o approfondire subito la versione completa del turk algorithm?

A: Sì, testiamo prima questo

Bene, integriamo `find_closest_in_chunk` al posto di `find_min_in_chunk` in `extract_from_chunk`.

```
c

#include "push_swap.h"

static int    int_sqrt(int n)
{
    int    i;

    i = 1;
    while (i * i <= n)
        i++;
    return (i - 1);
}

static t_node    *find_closest_in_chunk(t_node *a, int max_rank, int size)
{
    t_node    *best;
    t_node    *tmp;
    int        best_dist;
    int        pos;
    int        dist;

    best = NULL;
    best_dist = size + 1;
    tmp = a;
    pos = 0;
    while (tmp)
    {
        if (tmp->rank < max_rank)
        {
            dist = pos;
            if (dist > size / 2)
                dist = size - dist;
            if (dist < best_dist)
            {
                best = tmp;
                best_dist = dist;
            }
        }
        tmp = tmp->next;
        pos++;
    }
    return (best);
}
```

```
}

static int      extract_from_chunk(t_node **a, t_node **b, int *count,
                                   int current_chunk, int chunk_size)
{
    t_node  *target;
    int      dist;
    int      size;
    int      max_rank;

    max_rank = (current_chunk + 1) * chunk_size;
    size = stack_size(*a);
    target = find_closest_in_chunk(*a, max_rank, size);
    if (!target)
        return (0);
    dist = position_of_node(*a, target);
    move_to_top(a, dist, size, count);
    push_b(a, b, count);
    return (1);
}

int      algo_medium(t_node **a, t_node **b)
{
    int     count;
    int     size;
    int     chunk_size;
    int     current_chunk;
    int     num_chunks;

    count = 0;
    size = stack_size(*a);
    num_chunks = int_sqrt(size);
    if (num_chunks <= 0)
        num_chunks = 1;
    chunk_size = size / num_chunks + 1;
    current_chunk = 0;
    while (*a)
    {
        if (!extract_from_chunk(a, b, &count, current_chunk, chunk_size))
            current_chunk++;
    }
    while (*b)
        push_a(a, b, &count);
}
```

```
        return (count);  
    }
```

Cosa è cambiato

`find_closest_in_chunk` calcola la **distanza minima** (considerando sia `ra` che `rra`) per ogni candidato nel chunk corrente, e scegli quello con distanza minore — non più il rank assoluto più basso.

Ricompila e testa di nuovo con 500 numeri a disordine 0.3 — dimmi il nuovo conteggio di operazioni!

non mi sta dando i numeri in ordine, con findclosest non funziona

Il problema è concettuale: cambiando la strategia di selezione (dal rank minimo alla distanza minima), **rompi l'invariante** che garantiva l'ordine corretto in `b`.

Perché succedeva prima (con `find_min_in_chunk`)

Dentro ogni chunk, prendevi sempre il rank PIÙ BASSO disponibile.
Quindi gli elementi finivano in `b` sempre in ordine crescente di rank.
`b` (dal top al bottom) = ordine decrescente → `pa` ripristinava l'ordine corretto.

Perché si rompe con `find_closest_in_chunk`

Ora prendi l'elemento PIÙ VICINO al top, non il più piccolo.
Due elementi nello stesso chunk possono finire in `b` in un ordine diverso
da quello dei loro rank → quando fai `pa`, NON ottieni più l'ordine corretto!

Esempio concreto del problema

chunk corrente contiene rank 3 e rank 5
rank 3 è in fondo allo stack (lontano)
rank 5 è vicino al top

```
find_closest prende rank 5 PRIMA di rank 3
→ in b: rank 5 va sotto rank 3 (impilati in ordine sbagliato)
→ quando fai pa, l'ordine finale NON sarà corretto
```

La soluzione corretta

Devi garantire che **dentro lo stesso chunk**, gli elementi vengano comunque estratti in ordine di rank crescente — ma puoi scegliere liberamente **tra chunk diversi** o usare distanza come criterio **secondario** solo quando i rank sono dello stesso "livello" di urgenza.

Il vero Turk algorithm gestisce questo diversamente: non guarda solo "il chunk attuale", ma cerca tra **tutti gli elementi non ancora spostati**, quello con il rank più basso che sia anche raggiungibile in modo efficiente, mantenendo però la costruzione di **b** già ordinata (push in cima o in fondo a seconda della posizione, non sempre push_b semplice).

Torniamo alla versione che funzionava (find_min_in_chunk) e ottimizziamo diversamente

Il vero risparmio nel Turk algorithm non viene dal *quale elemento* scegli dentro il chunk, ma da **come costruisci b**: invece di fare sempre **push_b** in testa, decidi se metterlo in testa o in fondo a **b** in base alla sua posizione relativa — così quando reinserisci in **a**, hai meno rotazioni anche lì.

Vuoi che ripristiniamo **find_min_in_chunk** (che dava risultati corretti) e poi vediamo insieme la vera tecnica di posizionamento intelligente in **b**?

sí ho ripristinato, facciamo il posizionamento intelligente di b

Bene. Ripristinato **find_min_in_chunk**, ora vediamo la vera ottimizzazione del Turk algorithm: il posizionamento intelligente in **b**.

Il problema attuale

Quando fai sempre **push_b**, l'elemento finisce sempre in testa a **b**. Poi per rimetterlo in **a**, devi sempre fare **pa** dalla testa — ma se gli elementi in **b** non sono ordinati in modo "comodo", il riposizionamento finale in **a** può costare molte rotazioni extra.

L'idea — costruire **b** già "ordinato per rotazione"

In **b**, vuoi che gli elementi siano disposti in modo che, quando li rimetti in **a**, **siano già vicini alla posizione corretta** — minimizzando le rotazioni sia in **b** (per scegliere dove inserirlo) sia in **a** (al momento del **pa**).

Quando un elemento esce da 'a' ed entra in 'b':

- se il suo rank è MAGGIORE del top di b → mettilo in cima (pb normale)
- se il suo rank è MINORE del top di b → prima ruota b finché il punto di inserimento giusto è in cima, POI pb

In pratica: mantieni **b** **ordinato** (decrescente dal top, ad esempio), così quando rimetti tutto in **a** con **pa**, l'operazione è quasi gratuita.

Come si traduce in codice

```
c
static void    insert_into_b(t_node **a, t_node **b, int *count)
{
    int        size_b;
    int        pos;

    size_b = stack_size(*b);
    pos = find_insert_position(*b, (*a)->value, size_b);
    move_b_to_top(b, pos, size_b, count);
    push_b(a, b, count);
}
```

find_insert_position — dove inserire mantenendo **b** ordinato

```
c
static int    find_insert_position(t_node *b, int value, int size_b)
{
    int        pos;
```

```

    pos = 0;
    while (b && b->value > value)
    {
        b = b->next;
        pos++;
    }
    (void)size_b;
    return (pos);
}

```

Questo conta quante posizioni servono per trovare il punto giusto di inserimento (mantenendo **b** ordinato in modo decrescente dal top).

move_b_to_top — porta quella posizione in cima a **b**

```

c
static void    move_b_to_top(t_node **b, int pos, int size, int *count)
{
    if (pos <= size / 2)
    {
        while (pos > 0)
        {
            rotate_b(b, count);
            pos--;
        }
    }
    else
    {
        while (pos < size)
        {
            rrotate_b(b, count);
            pos++;
        }
    }
}

```

Come cambia **extract_from_chunk**

```

c
static int      extract_from_chunk(t_node **a, t_node **b, int *count,
                                   int current_chunk, int chunk_size)
{
    t_node  *target;
    int      dist;
    int      size;
    int      max_rank;

    max_rank = (current_chunk + 1) * chunk_size;
    target = find_min_in_chunk(*a, max_rank);
    if (!target)
        return (0);
    size = stack_size(*a);
    dist = position_of_node(*a, target);
    move_to_top(a, dist, size, count);
    insert_into_b(a, b, count);    // ← invece di push_b semplice
    return (1);
}

```

E alla fine, il reinserimento in **a** diventa quasi banale

Se **b** è mantenuto ordinato (decrescente dal top), allora **pa** ripetuto N volte ti dà direttamente l'ordine corretto in **a** — esattamente come prima, ma ora **anche dentro b** hai speso meno rotazioni **totali** grazie all'inserimento ordinato invece che caotico.

Hai bisogno che ti aiuti a contare le funzioni per file (probabilmente dovrai dividere ancora), o vuoi prima implementare e testare i numeri con questa versione?